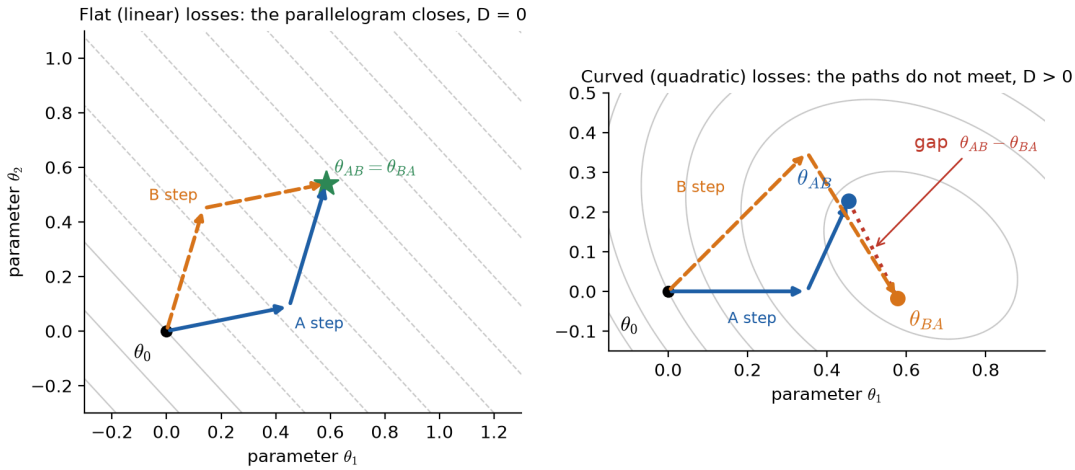
ONBOARDING NOTE · GROKKING PREDICTORS

The Commutator Defect, Explained from Scratch

What it measures, why it might warn us before a network groks, and exactly how we will test it in your benchmark



Solid blue: step on batch A, then on batch B. Dashed orange: B first, then A. Grey lines: contours of $L_A + L_B$. (Big step size η used only so the gap is visible.)

Prepared for: Jonathan John

Thesis: "A Unified Benchmark of Grokking Predictors in Neural Networks"

Prepared by: Grok Bot · 1 November 2026

Level: no prior knowledge of the commutator defect is assumed

1. The idea in one paragraph
2. Background refresher
3. Intuition: does the order of steps matter?
4. The definition, step by step
5. Why it might predict grokking
6. Sources, and what each one gives us
7. How it fits your experiment
8. The stage-by-stage plan
9. Honest caveats
10. Glossary and one-page cheat sheet

1. The idea in one paragraph

Let us begin with a simple experiment. Take your network at some moment during training. Pick two small groups of training examples. We call them **batch A** and **batch B**. Now make a copy of the weights and do two small trials on it. In the first trial, take one small gradient step using A, and then one using B. In the second trial, go back to the same starting weights. This time take the B step first, and then the A step. If the loss surface near the current weights is flat, the order does not matter. Both trials end at the same place. If the surface is curved, the two trials end at slightly different places. The **commutator defect**, written D, is the size of this gap. We divide it by the sizes of the two steps, so that it does not depend on how big the steps were. In other words, D is a cheap "curvature meter" for the loss landscape. Yongzhong Xu (2026) reports that in grokking runs, this meter rises sharply *before* test accuracy rises. If this is also true in our setting, D can act as an early-warning signal for grokking. Our job is to test this claim fairly, with rules fixed in advance.

> **THE WHOLE DOCUMENT IN THREE LINES**
>
> **What:** D = how much "A then B" and "B then A" disagree, per unit of step size.
> **Why it may matter:** Xu reports that D rises 600–1600 steps before grokking in his setup.
> **Our test:** compute D on your 5 saved Nanda-style runs, apply Xu's onset rule, and score it against frozen pass criteria.

2. Background refresher

2.1 What grokking is

In a grokking experiment, a small network learns a simple rule, such as $(a + b) \bmod p$. We train it on only some of the possible (a, b) pairs. We then test it on the remaining pairs. Let us see what happens, step by step.

1. **Memorisation.** Training accuracy reaches 100% quite early. But test accuracy stays near chance. The network has only memorised the answers it has seen. It is like a student who has learnt the answer key by heart, without understanding the method.
2. **A long plateau.** For thousands of further epochs, the accuracy plots do not seem to change at all.
3. **Sudden generalisation.** Then, much later, test accuracy suddenly shoots up to nearly 100% in a short time. The network has now found the real rule. This late, sudden jump is called **grokking**.

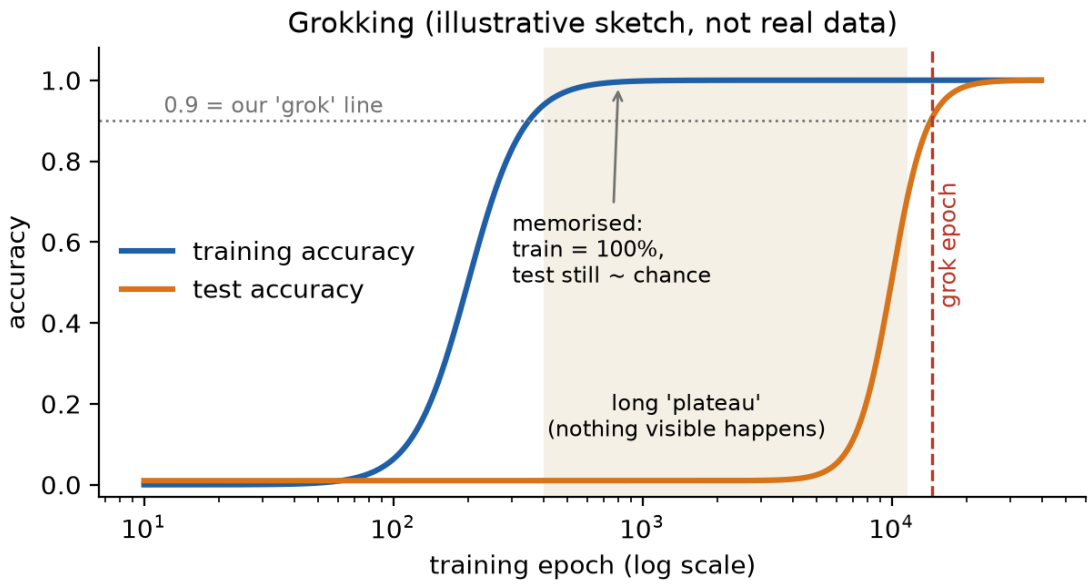


Figure 1. The shape of a grokking run (a sketch). In your benchmark, the "grok epoch" is the first epoch at which test accuracy is above 0.9 (red dashed line).

Now we can state the main practical question of your thesis. **During the plateau, when the accuracy curves are flat, can any measurement tell us that grokking is coming, and roughly when?** Each such measurement is called a "grokking predictor". The commutator defect is one such candidate.

2.2 Gradient descent in one picture

A network has many weights. We collect all of them into one long list of numbers. This list is called the **parameter vector**, and we write it as θ (read as "theta"). Your transformer has 210,432 numbers in θ. The **loss** $L(\theta)$ is a single number. It tells us how wrong the network is. Think of the loss as the height of the ground on a landscape. Every possible θ is a location, and the loss is the height of the ground at that location. Training simply means walking downhill.

The **gradient** $g(\theta)$ is an arrow. It points in the direction in which the ground rises most steeply, that is, steepest *uphill*. So, to go downhill, we step in the opposite direction. One plain gradient-descent step is:

$$\theta_{\text{new}} \;=\; \theta \;-\; \eta\, g(\theta)$$

Here η (read as "eta") is the **step size**, also called the learning rate. It is a small positive number, like 0.001. In other words, at each step we move a little bit in the downhill direction.

2.3 Mini-batches

The full loss is an average over all the training examples. Often, we estimate the gradient using only a small random group of examples. Such a group is called a **mini-batch**. Two different mini-batches, A and B, give two slightly different gradients, g_A and g_B. This is because each batch "sees" different examples. Note that this difference is essential for the commutator defect. If A and B were the same batch, the two trials in Section 1 would be exactly the same, and the defect would be exactly zero.

ONE FACT TO REMEMBER FOR LATER

Your training runs were **full-batch**. This means every AdamW step used all 3,830 training examples. So, in your runs, **1 epoch = 1 optimiser step**. For the defect measurement, however, we still need two *different* batches. So we will draw A and B (512 examples each) from the training set, only for the measurement. Training itself is not changed in any way.

2.4 Curvature, the Hessian, in plain words

The gradient tells you the slope. **Curvature** tells you how fast the slope changes as you move. Let us take two everyday examples. A flat, tilted roof has a slope, but zero curvature: wherever you walk on it, the slope is the same. A bowl has curvature: as you move, the slope keeps changing. The mathematical object that stores all the curvature information is called the **Hessian**, written H. It is a square table of numbers, that is, a matrix. Suppose you make a small move v. Then Hv tells you how much the gradient changes because of that move. Note that we will never build H for the real network. It would have $210{,}432 \times 210{,}432$ entries! The beauty of the defect is that it senses the curvature *without* ever building H.

3. Intuition: does the order of two steps matter?

3.1 The walking analogy

Imagine that you are standing in a big, flat field. Walk 100 metres North, and then 100 metres East. Mark the spot. Now go back to the start. This time, walk 100 metres East first, and then 100 metres North. You will land on **the same spot**. So, on flat ground, the order of two moves does not matter. Mathematicians say that the moves **commute**. (Just as $2 + 3 = 3 + 2$.)

Now let us do the same thing on a globe, with large distances. Start on the equator. Walk 6,000 km North. Then walk 6,000 km East along that line of latitude. Note that far up North, the circles of latitude are small. So 6,000 km East takes you a long way round the circle. Now go back to the start. Walk East first along the equator. Here the circle is big, so 6,000 km takes you a shorter way round. Then walk North. You land at a **different point**. So, on curved ground, the order matters. The moves **do not commute**.

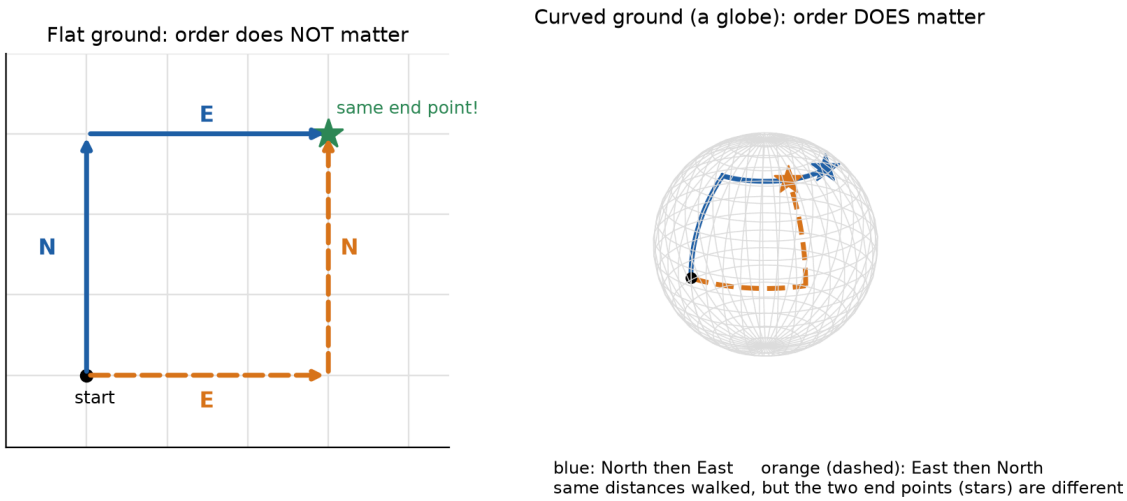


Figure 2. Left: on flat ground, "North then East" and "East then North" end at the same point. Right: on a sphere, they do not. The size of the mismatch tells us how curved the ground is.

> **KEY IDEA**
>
> The amount by which two moves fail to commute is a measure of curvature. That is all the commutator defect is. It is the same walking experiment, done in weight space, using gradient steps instead of walks.

3.2 The same thing with gradient steps

In training, a "move" is a gradient step. But there is one twist, which makes the picture more interesting. The direction of a gradient step *depends on where you are standing*. After you take the A step, you are at a new place. Batch B's gradient at this new place may point in a slightly different direction from where it pointed at the start. Let us look at the two possible cases.

- Suppose each batch's loss is a flat, tilted plane (its contours are straight lines). Then its gradient is the same everywhere. So the second step does not care where the first step left you. The two routes form a closed parallelogram. **Defect = 0.**
- Now suppose the losses are curved. Then the direction of the second step changes, depending on the first step. The parallelogram does not close. **Defect > 0.**

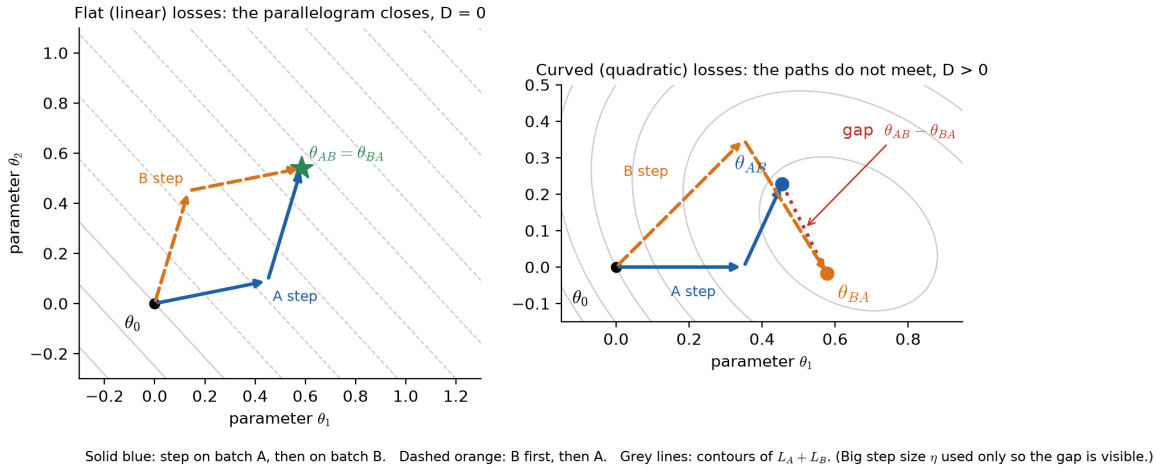


Figure 3. A toy example with only 2 parameters. Left: linear (flat) losses; the two routes meet exactly. Right: quadratic (curved) losses; the routes end at θ_{AB} and θ_{BA}. The red dotted segment is the gap that the defect measures. Note that the step size here is very large ($\eta = 0.35$), only so that you can see the gap. In practice we use $\eta = 0.001$, and the gap is microscopic. This is why we divide by the step sizes (Section 4.3).

Note one subtle point here. Curvature is needed for a non-zero defect, but it is not the only ingredient. The two batches must also "disagree". In other words, they must have different gradients or different curvatures. If A and B are the same batch, then $g_A = g_B$, and the defect is zero even on a curved landscape. So D really measures **curvature as felt through the disagreement between two batches**.

4. The definition, step by step

4.1 The two routes

We start from the current weights, θ_0. The route "A then B" is:

$$\theta_{AB} \;=\; \theta_0 \;-\; \eta\, g_A(\theta_0) \;-\; \eta\, g_B\!\left(\theta_0 - \eta\, g_A(\theta_0)\right)$$

Let us read it from left to right. Start at θ_0. Take the A step, $-\eta\, g_A(\theta_0)$. You are now at $\theta_0 - \eta\, g_A(\theta_0)$. Compute B's gradient *at this new point,* and take the B step. The route "B then A" is simply the mirror image:

$$\theta_{BA} \;=\; \theta_0 \;-\; \eta\, g_B(\theta_0) \;-\; \eta\, g_A\!\left(\theta_0 - \eta\, g_B(\theta_0)\right)$$

4.2 The defect

$$D \;=\; \frac{\|\,\theta_{AB} - \theta_{BA}\,\|}{\|\,\eta\, g_A(\theta_0)\,\| \;\cdot\; \|\,\eta\, g_B(\theta_0)\,\|}$$

Symbol	Meaning in plain words
θ_0	The weights of the network at the checkpoint we are examining (all 210,432 numbers).
A, B	Two different random mini-batches of training examples (512 each in our spec).
$g_A(\theta), g_B(\theta)$	Gradient of the training cross-entropy loss computed on batch A (or B) at weights θ. A vector with one entry per weight.
η	The probe step size. Fixed at 10^{-3}. It is only used inside the measurement; it is not the training learning rate (they happen to be equal in Xu's main setting).
θ_{AB}, θ_{BA}	Where you end up after the two routes.
$\|v\|$	The ordinary length of a vector: square every entry, add them, take the square root (the "L2 norm").
$\|\eta\, g_A\|, \|\eta\, g_B\|$	The lengths of the first A step and the first B step, both measured at θ_0.
D	The (scale-normalised) commutator defect. A single non-negative number. 0 means "order does not matter here".

Cost. Each value of D needs four gradient computations. These are: $g_A(\theta_0)$, $g_B(\theta_0)$, g_B at the A-moved point, and g_A at the B-moved point. Each computation is one forward pass and one backward pass on 512 examples. One pair (A, B) gives a noisy answer. So we repeat the measurement with **K = 5** fresh pairs, and report the **median** (the middle value). We also report the 25th and 75th percentiles.

4.3 Why divide? (normalisation)

The raw gap $\|\theta_{AB} - \theta_{BA}\|$ depends strongly on the step size. In Section 4.4 we will see that it shrinks like η^2. In other words, if you halve η, the gap becomes four times smaller. The gap also grows when the gradients are bigger. Now look at the denominator, $\|\eta\, g_A\| \cdot \|\eta\, g_B\|$. It also contains exactly η^2 and the two gradient lengths. So, when we divide, the η^2 cancels. Therefore, as long as η is small, D does not depend on our choice of η. It becomes a property of the landscape, not of our measuring stick.

Analogy. Suppose you want to compare how curved two roads are. You would not just compare how far a car drifts sideways, because that depends on how far the car drove. Instead, you would divide by the distance driven. The same idea is used here.

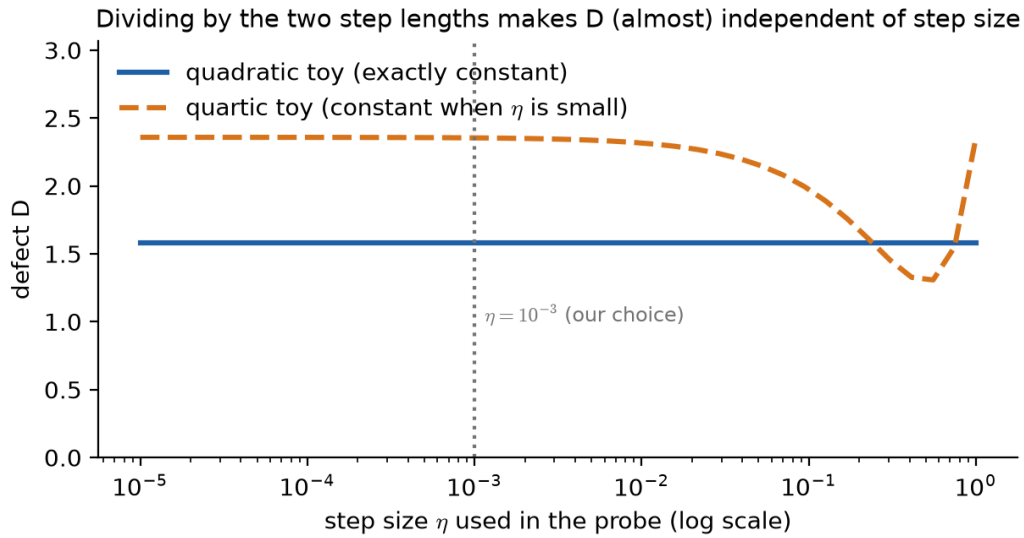


Figure 4. Computed with code on two toy losses. For a pure quadratic loss, D is exactly the same for every η. For a non-quadratic (quartic) toy loss, D is flat for small η, and it drifts only when η becomes large. At $\eta = 10^{-3}$ we are safely in the flat region. This is the basis of the "halve η, and D should stay roughly the same" sanity check in Section 8.

4.4 Where the name comes from: the first-order expansion

Let us now see what the gap is made of. We need only one tool: a first-order Taylor expansion. In simple words, it says: *if you move a tiny bit, the gradient changes by (curvature) × (how far you moved)*. For batch B, moving by $-\eta\, g_A$, this gives:

$$g_B(\theta_0 - \eta\, g_A) \;\approx\; g_B(\theta_0) \;-\; \eta\, H_B\, g_A$$

Here H_B is the Hessian (curvature matrix) of batch B's loss at θ_0. Now let us put this into the formula for θ_{AB}:

$$\begin{aligned}\theta_{AB} &\approx \theta_0 - \eta\, g_A - \eta\,\big[\, g_B - \eta\, H_B\, g_A \,\big] \\ &= \theta_0 - \eta\,(g_A + g_B) + \eta^2\, H_B\, g_A\end{aligned}$$

and, in the same way (by symmetry),

$$\theta_{BA} \approx \theta_0 - \eta\,(g_A + g_B) + \eta^2\, H_A\, g_B \qquad (\text{all } g,\ H \text{ evaluated at } \theta_0)$$

Note that the first-order parts, $-\eta\,(g_A + g_B)$, are the same in both routes. This is because addition does not depend on order. So they cancel when we subtract. What remains is:

$$\theta_{AB} - \theta_{BA} \;\approx\; \eta^2 (H_B\, g_A \;-\; H_A\, g_B)$$

In words, this expression is "B's curvature acting on A's gradient, minus A's curvature acting on B's gradient". Mathematicians call it the **Lie bracket** of the two gradient fields. The Lie bracket is the standard way to measure how far two "flows" are from commuting. In algebra, an expression of the form $XY - YX$ is called a **commutator**. This is where the name **commutator defect** comes from. It is the "defect", or failure, of two gradient steps to commute. Now let us put this into the definition of D:

$$D \approx \frac{\eta^2 \|H_B g_A - H_A g_B\|}{\eta^2 \|g_A\| \|g_B\|} = \frac{\|H_B g_A - H_A g_B\|}{\|g_A\| \|g_B\|}$$

Look carefully at the right-hand side. Note that η has disappeared. This is the step-size independence that we wanted. What is left has **curvature on top** and **gradient sizes underneath**. Please remember this shape; it will matter in the caveats (Section 9).

> **TWO QUICK CHECKS OF UNDERSTANDING**
>
> (1) Flat landscape: $H_A = H_B = 0$. So the bracket is 0, and $D = 0$. ✓
> (2) Same batch twice: $g_A = g_B$ and $H_A = H_B$. So $Hg - Hg = 0$, and $D = 0$. ✓ (In code this holds exactly, not just approximately, because both routes perform identical operations.)

Theory background. The order of SGD steps matters only at second order in η; this is the η^2 term above. This fact is also central in the work of Smith, Dherin, Barrett & De (2021). They use this kind of expansion to explain the implicit regularisation of SGD. We use their paper only as background for this mathematical fact, not as evidence about grokking.

4.5 Worked toy example, by hand

Let us now work out one example completely, by hand. We take only two parameters, $\theta = (\theta_1, \theta_2)$. Each "batch" has its own bowl-shaped loss:

$$L_A(\theta) = \tfrac{1}{2}(\theta - c_A)^\top H_A (\theta - c_A), \qquad L_B(\theta) = \tfrac{1}{2}(\theta - c_B)^\top H_B (\theta - c_B)$$

$$H_A = \begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix}, \quad c_A = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \qquad H_B = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}, \quad c_B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

For such a bowl, the gradient is simply $g_A(\theta) = H_A(\theta - c_A)$. Similarly, $g_B(\theta) = H_B(\theta - c_B)$. We start at $\theta_0 = (0, 0)$ and use $\eta = 0.1$.

Step 1. Gradients at the start

$g_A(\theta_0) = H_A\,(0-1,\ 0-0) = H_A\,(-1,\ 0) = (-1,\ 0)$, length 1
$g_B(\theta_0) = H_B\,(0-0,\ 0-1) = H_B\,(0,\ -1) = (-1,\ -1)$, length $\sqrt{2} \approx 1.414$

Step 2. Route A then B

after A: $\theta_0 - 0.1\,(-1,\ 0) = (0.10,\ 0.00)$
g_B there: $H_B\,(0.10,\ -1.00) = (-0.80,\ -0.90)$
$\theta_{AB} = (0.10,\ 0.00) - 0.1\,(-0.80,\ -0.90) = (0.18,\ 0.09)$

Step 3. Route B then A

after B: $\theta_0 - 0.1\,(-1,\ -1) = (0.10,\ 0.10)$
g_A there: $H_A\,(-0.90,\ 0.10) = (-0.90,\ 0.30)$
$\theta_{BA} = (0.10,\ 0.10) - 0.1\,(-0.90,\ 0.30) = (0.19,\ 0.07)$

Step 4. Gap and defect

$\theta_{AB} - \theta_{BA} = (-0.01,\ 0.02)$, length $\sqrt{0.0005} \approx 0.02236$
$\|\eta\, g_A\| \cdot \|\eta\, g_B\| = 0.1 \times 0.1414 = 0.01414$
$D = 0.02236 \,/\, 0.01414 \approx 1.581$

Now let us check this answer using the Lie-bracket formula. We have $H_B\, g_A = (-2, -1)$ and $H_A\, g_B = (-1, -3)$. Their difference is $(-1, 2)$. Multiplying by $\eta^2 = 0.01$ gives $(-0.01, 0.02)$. This

matches exactly. (Note that for pure quadratics, the "≈" becomes an exact "=", because the Hessian is the same everywhere.) Also, $D = \sqrt{5}/(1 \times \sqrt{2}) = \sqrt{2.5} \approx 1.581$ for *any* step size. This is the flat blue line in Figure 4. Code run on the box confirms the value 1.5811.

5. Why it might predict grokking (Xu's hypothesis)

5.1 The story Xu tells

Xu (2026, arXiv:2602.16746) studies small transformers that learn modular arithmetic. Let us see his picture, in simple words, in three stages.

1. **Confinement.** After memorisation, the weights move mostly along a very narrow, almost one-dimensional path in weight space. (He finds that one principal direction explains 68–83% of the weight movement.)
2. **Curvature builds up sideways.** While the weights slowly crawl along this path, the landscape becomes more and more sharply curved in the directions *across* the path. These sideways directions are called "transverse" directions. The commutator vectors point almost entirely in these sideways directions. This is the meaning of "transversely curved" in his title.
3. **Escape.** Finally, the trajectory escapes from this region into the generalising solution, and test accuracy jumps.

Note that the curvature build-up (stage 2) happens *before* the escape (stage 3). So a curvature meter should start rising before test accuracy rises. The commutator defect is exactly such a meter. Xu reports that in grokking runs, the defect grows 10–1000× more than in non-grokking controls, and that the rise begins before generalisation.

5.2 Turning a rising curve into a single "event": the onset rule

To compare predictors fairly, each predictor must give one **event epoch** per run. This is the epoch where we say "the predictor fires here". For the defect, we use Xu's own onset rule. Let $\tilde{D}(t)$ be the median defect (over the K pairs) at measurement t.

$$t_{\text{onset}} = \text{first } t \text{ such that } \tilde{D}(t) > 10\,b \text{ and } \tilde{D}(t) > 20, \qquad b = \text{median}\{\tilde{D}(t_1), \tilde{D}(t_2), \tilde{D}(t_3)\}$$

In other words, the rule works in two steps. First, find an **early baseline** b. This is the median of the first three measurements. Second, find the **first** measurement where the median defect is **more than ten times** this baseline **and also more than 20**. That measurement is the onset. Why do we need two conditions? Let us see.

- The condition "$10\times$ baseline" asks for a big rise relative to the start. So a run that simply starts high does not fire immediately.
- The condition "> 20" is an absolute floor. Suppose the baseline is tiny, say 0.3. Then $10\times$ is only 3, and normal noise could cross it. The floor stops such false alarms.

```
Worked example (median D at successive measurements):
  1.2, 0.9, 1.1, 1.5, 3.0, 8.0, 14, 25, 60, ...
  baseline b = median(1.2, 0.9, 1.1) = 1.1      →  10b = 11
  need D̃ > 11 AND D̃ > 20:   14 fails (not > 20),  25 passes
  → onset = the 8th measurement (the one with value 25)

Floor example: b = 0.3, so 10b = 3. A value of 5 beats 10b but not 20 → no onset yet.
```

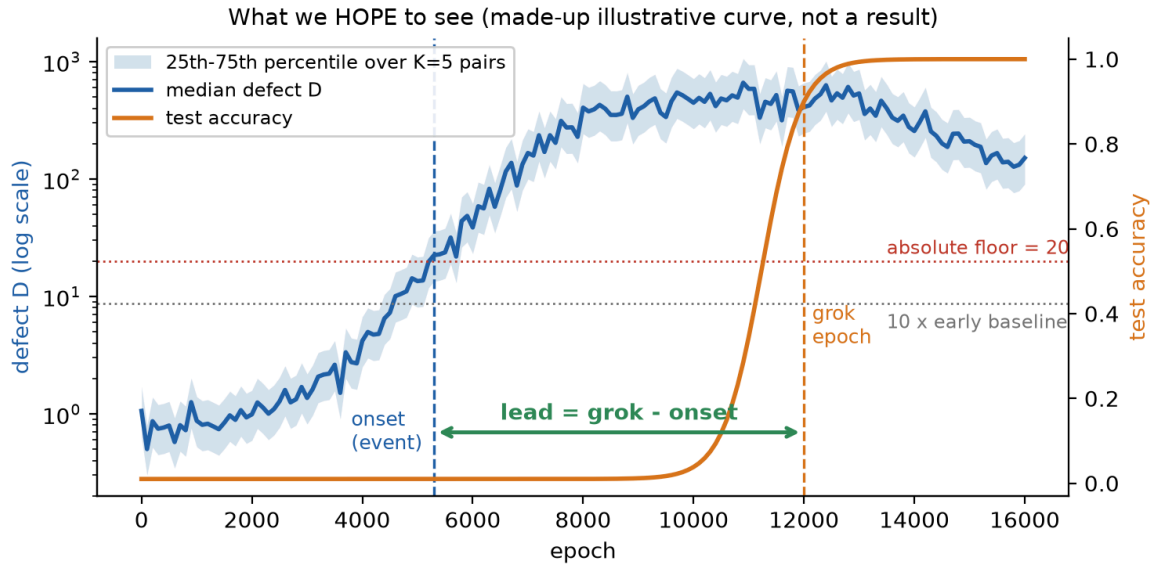


Figure 5. What a success would look like (a made-up curve, for illustration only). The median defect (blue, log scale) climbs from about 1 to the hundreds. The shaded band is the 25th–75th percentile over the 5 pairs. The onset fires where the curve first crosses both dotted lines. Test accuracy (orange) jumps later. The gap between the two is called the **lead**.

5.3 What Xu actually found (main paper)

Xu's main setting uses modular arithmetic mod 97, with learning rate 10^{-3}. He studied 12 grokking runs (4 operations × 3 seeds). In these runs, the defect onset came **600 to 1600 steps** before the step at which test accuracy reached 90% (the mean was about 1,117 steps). In every one of the 12 runs, the onset came first. This consistent ordering is the result that we are trying to reproduce, in a different setup.

6. Sources, and what each one gives us

We rely on only three papers. We do not use any other results.

6.1 Main source

Yongzhong Xu (2026). "Low-Dimensional and Transversely Curved Optimization Dynamics in Grokking." arXiv:2602.16746. Code: `github.com/skydancerosel/grokking-integrability`.

Item	Xu's main setting
Task	Binary operations mod 97 (add, sub, mul, $a^2 + b^2$ grok; two harder ones do not), 50/50 train/test split
Model	2-layer transformer encoder, $d_{\text{model}} = 128$, 4 heads, ≈ 290k parameters
Training	AdamW, lr 10^{-3}, weight decay 1.0 (0.0 for controls), $\beta = (0.9,\ 0.98)$, mini-batch 512, gradient clipping 1.0
Defect probe	Formula of Section 4; $\eta = 10^{-3}$; plain gradient steps on cross-entropy; all parameters; float32; measured every 100 steps; median over K pairs
Onset rule	Median $D > 10\times$ median of first 3 measurements AND > 20
Grok definition (for lead)	First step with test accuracy ≥ 90%
Key result	Onset preceded 90% test accuracy by 600–1600 steps in all 12 grokking runs

SMALL INCONSISTENCY TO BE AWARE OF

Note this small point. The paper text (Section 3.2 of the version I read) says K = 9 samples per measurement. However, the released code mostly uses `COMM_K = 5`, and the follow-up paper clearly uses K = 5. Our spec uses K = 5. This does not change the formula or the onset rule. It only affects how noisy each median is. It is worth one sentence in your methods chapter.

6.2 Extra source (we borrow only one thing)

Yongzhong Xu (2026). "Early-Warning Signals of Grokking via Loss-Landscape Geometry." arXiv:2602.16967. This paper uses the same defect formula and the same onset rule ($10\times$ baseline and floor 20) on other tasks (SCAN and Dyck-1). The **only thing we borrow** from it is its way of reporting. Along with the median over the K = 5 pairs, we also record the **25th and 75th percentiles**. This lets us see how noisy each measurement is.

6.3 Theory background

Samuel L. Smith, Benoit Dherin, David G. T. Barrett & Soham De (2021). "On the Origin of Implicit Regularization in Stochastic Gradient Descent." ICLR 2021, arXiv:2101.12176. We use it only for one mathematical point: the order of SGD mini-batch steps matters at second order in the step size. This is exactly the η^2 term of Section 4.4.

7. How it fits your experiment

7.1 Your setup

Item	Your benchmark (Nanda-style)
Task	Modular addition, $p = 113$
Model	1-layer transformer, 4 heads, 210,432 parameters
Training	Full-batch AdamW on 3,830 training examples (so 1 epoch = 1 step)
Runs	5 seeds, checkpoints saved every 100 epochs (401 checkpoints per seed)
Grok rule	First epoch with test accuracy > 0.9 (same 90% idea as Xu's main paper)
Grok epochs	seed 0: 12,987 · seed 1: 6,921 · seed 2: 9,510 · seed 3: 11,019 · seed 4: 23,747

7.2 Our defect spec, and why each choice was made

Choice	Value	Reason
Batches A and B	512 examples each, drawn **with replacement** from the train split	**Adaptation.** Training was full-batch, so there are no natural mini-batches. If we used the full batch for both, A = B and D would be exactly 0. 512 matches Xu.
Number of pairs	K = 5 fresh pairs per checkpoint	Matches Xu's code and the follow-up paper.
Summary	Median, plus 25th/75th percentiles	Median is the event signal; percentiles show spread (borrowed from paper 2).
Probe step size	$\eta = 10^{-3}$	Matches Xu. Small enough to be in the flat region of Figure 4.
Step type	Plain gradient steps on training cross-entropy	Matches Xu. Also means **no optimiser state is needed**: we do not need AdamW's moment estimates, only the weights in each checkpoint.
Which weights	All parameters	Matches Xu.
Precision / device	CPU, float64	**Adaptation.** The gap is of order $\eta^2 \approx 10^{-6}$ times the gradient sizes, so tiny; float64 keeps rounding error far below it. Apple's MPS backend has no float64, so CPU it is. (Paper 2 even mentions special handling when float32 becomes too imprecise.)
How often	Every 100 epochs (all 401 checkpoints)	Full-batch means 1 epoch = 1 step, so this matches Xu's "every 100 steps".
PRIMARY event	Xu's onset rule ($10\times$ baseline AND > 20)	Use the published rule unchanged; no tuning on our data.
If it never fires	Counted as a **miss** for that seed	A predictor that stays silent has not predicted anything.

IMPORTANT: THE PROBE DOES NOT TOUCH TRAINING

Let us be clear about this. For each saved checkpoint, we load the weights into a copy of the model. We do the two-route experiment and record D. Then we throw the moved weights away. Nothing is retrained. The checkpoints on disk are not changed.

7.3 The frozen PASS criteria, in plain words

Before we look at any defect curve, the pass/fail rules are fixed, or "frozen". Why? This stops us from adjusting the rules, even without meaning to, to make the predictor look good. For each seed, we define:

$$\text{lead} = t_{\text{grok}} - t_{\text{event}}, \qquad \text{relative lead} = \frac{t_{\text{grok}} - t_{\text{event}}}{t_{\text{grok}}}$$

The predictor PASSES only if **all three** of the following are true:

Criterion	Threshold	Plain meaning
lead_frac: fraction of seeds where event epoch < grok epoch − 100	≥ 0.8 (4 of 5 seeds)	"Does it usually fire early?" Firing must come at least one measurement interval (100 epochs) before grokking. A miss counts as no lead.
Spearman ρ between event epochs and grok epochs	≥ 0.9	"Does it get the *order* right?" Rank the seeds by when the predictor fires and by when they grok. $\rho = 1$ means the rankings agree perfectly. Seeds that grok later should fire later.
mean relative lead	between 0.05 and 0.9	"Is the warning a useful size?" Below 0.05 the warning comes so close to grokking that it is useless. Above 0.9 it fires almost at the start, which suggests it is reacting to something generic (e.g. memorisation) rather than to grokking.

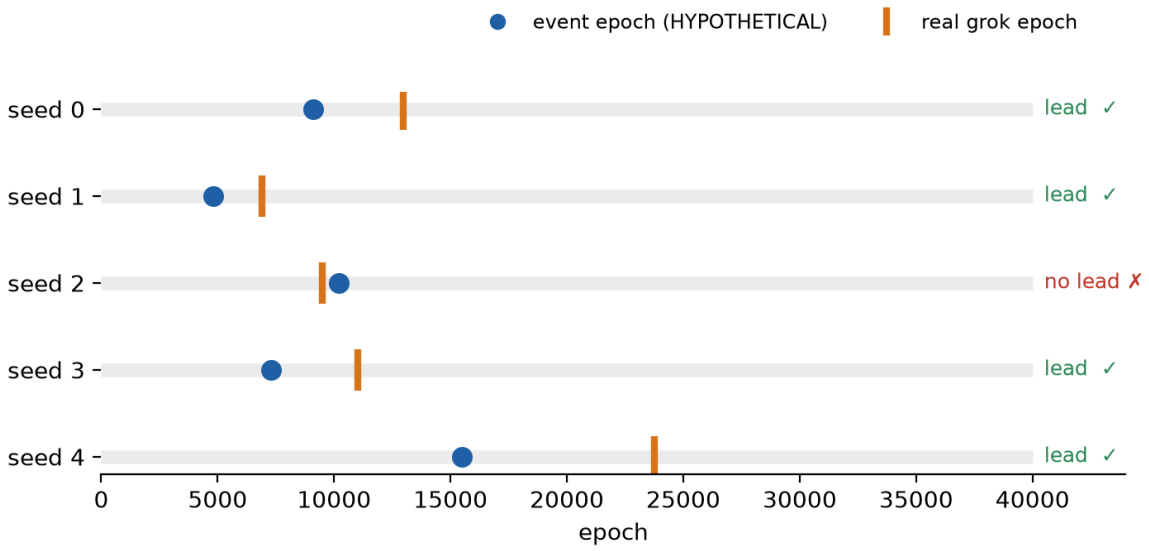


Figure 6. A practice scoring example. It uses your real grok epochs (orange ticks) and **invented** event epochs (blue dots). Note that the blue dots are not results.

```
Practice scoring with the invented events in Figure 6
  events  = [9100, 4800, 10200, 7300, 15500]
  groks   = [12987, 6921, 9510, 11019, 23747]
  lead?   = [yes, yes, NO (10200 > 9510-100), yes, yes]   → lead_frac = 4/5 = 0.80  ✓
  ranks   events: [3,1,4,2,5]   groks: [4,1,2,3,5]        → Spearman ρ = 0.70  ✗ (< 0.9)
  relative leads = [0.30, 0.31, -0.07, 0.34, 0.35]        → mean = 0.24  ✓
  verdict: FAIL (the ordering criterion is not met)
```

DECIDE BEFORE RUNNING (WRITE IT INTO CONTEXT.MD)

The criteria above are fixed. However, two small bookkeeping details should be written down clearly before any curve is seen. (1) How does a **missed** seed enter Spearman ρ and the mean relative lead? (For example: it is excluded, or it is treated as firing at the last checkpoint.) (2) Are negative relative leads (late firing, like seed 2 above) included in the average, as in the practice example? Whatever the existing benchmark code already does for other predictors should simply be reused, so that all predictors are treated the same way.

8. The stage-by-stage plan

1. **Freeze the rules in `context.md`.** Write down everything in Section 7: batch size and sampling, K, η, float64 on CPU, the every-100-epoch schedule, the onset rule ($10\times$ baseline of the first 3, floor 20), miss handling, the grok rule (test acc > 0.9), and the three PASS thresholds. Also record the random seed used for drawing A and B. After this, nothing changes.

2. **Write the predictor file.** This is a small module. Given a model and its training data, it computes D for K = 5 pairs. It returns the median, the 25th and 75th percentiles, and the per-pair values for the record. Then write a function that applies the onset rule to the whole series. It returns the event epoch, or "miss".

3. **Sanity checks** (do these before touching the real runs):
   - **Same batch twice gives zero.** Set A = B. Now the two routes do identical operations, so D must be exactly 0. This catches bugs where the weights are not reset between routes.
   - **Halving η keeps D roughly constant.** Compute D at $\eta = 10^{-3}$ and at $\eta = 5 \times 10^{-4}$ on a few checkpoints, using the same A and B. The two values should agree closely (Figure 4). If D doubles, the normalisation is wrong.
   - **Reproducible.** The same checkpoint with the same sampling seed should give the same numbers, twice in a row.
   - Optionally, use the 2-parameter toy of Section 4.5 as a unit test. The code should return 1.5811.

4. **Plug into the benchmark.** Register the predictor in `run_nanda_benchmark.py` (to compute and store the series for each seed). Also register it in `analyze_nanda_unified.py` (to apply the onset rule and the shared scoring).

5. **Run on the 5 seeds from saved checkpoints.** No retraining is needed. Let us estimate the cost: 401 checkpoints × 5 pairs × 4 gradient passes ≈ 8,000 passes per seed. That is about 40,000 passes in total. Each pass uses 512 examples, with a 210k-parameter model, in float64 on CPU. Expect **minutes to a few tens of minutes** overall. Time the first seed to get a real figure.

6. **Look at the raw curves first.** Before reading any score, plot the median D (with the 25–75% band) against epoch for each seed. Mark the grok epoch and the onset, just like Figure 5. Then ask: Does D rise? When? Is the band narrow? Does anything odd happen around memorisation? Also look at the gradient norms $\|g_A\|$, $\|g_B\|$ over time (see caveat 9.7).

7. **Score** using the frozen criteria: lead_frac, Spearman ρ, mean relative lead → PASS or FAIL.

8. **Log weak points.** Record every adaptation (full-batch → sampled batches, float64, K = 5). Note any surprises in the curves, and which caveats from Section 9 seem to matter. Remember, a FAIL is also a valid, publishable benchmark result.

9. Honest caveats: what could go wrong

In a fair benchmark, we expect some predictors to fail. So let us list the reasons why this one might fail. Then nobody will be surprised.

9.1 At lr 10^{-3}, the warning window in Xu's own data was modest

Xu's learning-rate sweep shows that the lead fraction (lead ÷ grok step) becomes smaller as the learning rate grows. It is about 95% at lr 3×10^{-5}. But it is only **43% at lr** 10^{-3} and **24% at lr** 3×10^{-3}. At the fastest rates, he says, the onset happens at the same time as grokking, instead of before it. Now, check your training learning rate in your config (10^{-3} is the usual choice in Nanda-style setups). If it is in this range, a short lead, or even a near miss, would agree with his own numbers.

9.2 Necessary but not sufficient

In Xu's main paper, the operations that never grok also showed defect growth of **30–50×** their baseline. Note that this is enough to pass a $10\times$ test. So a rise in D tells us "the landscape is reorganising". It does not tell us "grokking will happen". Xu himself calls the onset "necessary but not sufficient".

9.3 Separation from controls only in hindsight

Grokking runs finally reach much larger defects (Xu: 500–2000× baseline) than the controls. But he notes that this separation is visible only *retrospectively*, that is, when looking back. Early in training, when a warning would be useful, the curves of grokking and non-grokking runs overlap.

9.4 Weight decay matters

Runs of the same tasks **without weight decay** show a similar rise in defect, but they do not generalise. So the defect alone cannot be the whole story. Regularisation is another ingredient.

9.5 Limited evidence base

All the evidence from experiments comes from **single-author preprints**. As far as we know, they are not yet peer-reviewed, and there is **no independent replication**. In effect, your benchmark would be an independent test.

9.6 The floor of 20 may not transfer

The absolute floor (20) was chosen using Xu's model (2 layers, mod 97, real mini-batch training, float32). Your model is different: 1 layer, $p = 113$, full-batch training, and probe batches sampled from the full training set. So the typical size of D might be quite different here. The floor could be too high (it never fires, giving a miss), or too low (it fires on noise). We still keep it. This is because changing it after seeing the data would break the "frozen rules" principle. Any sensitivity analysis must be clearly labelled as secondary.

9.7 OUR OWN REASONING (NOT FROM THE PAPERS): D MAY RISE JUST BECAUSE GRADIENTS SHRINK

From Section 4.4, $D \approx (\text{curvature term}) \,/\, (\|g_A\| \cdot \|g_B\|)$. Now suppose the curvature stays the same, but both gradients become s times smaller. Then the top changes by a factor s, and the bottom by a factor s^2:

$$g \to s\,g \;\; (\text{curvature } H \text{ unchanged}) \quad \Longrightarrow \quad D \to \tfrac{s}{s^2}\,D = \tfrac{D}{s}$$

In other words, if the gradients shrink 100-fold, D grows 100-fold **with no change in curvature at all**. After memorisation, the training loss becomes very small, and so do the gradients. So a big rise in D could partly be a "gradients got small" effect, and not a "landscape got curved" effect. This is why Stage 6 asks you to also plot the gradient norms. If the rise of D mirrors the fall of $\|g\|$, interpret it with care.

9.8 Our adaptations are real differences

Full-batch training with sampled probe batches is not the same as Xu's mini-batch training. In his runs, A and B are like the batches that the optimiser actually used. In ours, they are only a measuring device. We think this is the most faithful choice available. But it should be clearly stated as an adaptation.

10. Glossary

Term	Simple meaning
Parameters, θ	All the weights of the network, viewed as one long list of numbers.
Loss, $L(\theta)$	One number measuring how wrong the network is. Think: height of the ground.
Cross-entropy	The standard loss for classification. Small when the network gives high probability to the right answer.
Gradient, g	Arrow pointing steepest uphill on the loss landscape. We step the other way.
Step size / learning rate, η	How far each step goes. Here the probe uses $\eta = 10^{-3}$.
Mini-batch	A small random group of training examples used to estimate the gradient.
Full-batch	Every step uses the entire training set (your training runs).
Sampling with replacement	Each example in a batch is drawn independently; the same example may appear twice.
AdamW	A popular optimiser with momentum-like memory and weight decay. Not used inside the probe.
Weight decay	A pull of all weights towards zero at every step; known to matter for grokking.
Curvature	How quickly the slope changes as you move.
Hessian, H	The matrix that stores the curvature. Hv = how the gradient changes for a small move v.
Commute	Order does not matter ($2 + 3 = 3 + 2$).
Commutator	$XY - YX$: the leftover when order is swapped. Zero if X and Y commute.
Lie bracket	The commutator of two flows/vector fields; here $\eta^2(H_B\, g_A - H_A\, g_B)$.
Commutator defect, D	$\|\theta_{AB} - \theta_{BA}\|$ divided by $\|\eta\, g_A\| \cdot \|\eta\, g_B\|$.
Taylor expansion (first order)	"For a tiny move, the change ≈ rate of change × size of move."
Norm, $\|v\|$	Length of a vector (square root of the sum of squares).
Median, percentile	Median = middle value. 25th/75th percentile = values below which 25%/75% of the data lie.
Baseline, b	Median of the first three median-D measurements.
Onset / event epoch	First measurement with median $D > 10b$ and > 20.
Grok epoch	First epoch with test accuracy > 0.9.
Lead, relative lead	grok − event; and that divided by grok.
Spearman ρ	Correlation of rankings. 1 = same order, 0 = unrelated, −1 = reversed.
Transverse	Sideways: directions across the path the weights are moving along.
float32 / float64	Number precision: about 7 vs about 16 significant digits.
Checkpoint	Saved copy of the weights at a given epoch.
Frozen criteria	Pass/fail rules written down before seeing results, never changed afterwards.

One-page cheat sheet

What it is

Start from weights θ_0. Take a gradient step on batch A, then on B. Separately, take B first, then A. Measure the gap, and divide by the two step lengths.

$$D = \frac{\| \theta_{AB} - \theta_{BA} \|}{\| \eta\, g_A(\theta_0) \| \cdot \| \eta\, g_B(\theta_0) \|}$$

Why "commutator"

$\theta_{AB} - \theta_{BA} \approx \eta^2 (H_B\, g_A - H_A\, g_B)$: this is the Lie bracket. If the landscape is flat, or if A = B, then $D = 0$. Dividing removes η^2, so D does not depend on step size (for small η).

Intuition

North-then-East = East-then-North on flat ground, but not on a globe. D is a curvature meter that never builds the Hessian.

Hypothesis (Xu 2026)

After memorisation, the weights crawl along a narrow path while sideways curvature builds up. D climbs. Then the model escapes and groks. Main paper: onset came 600–1600 steps before 90% test accuracy in 12/12 grokking runs (mod 97, lr 10^{-3}).

Onset rule (event)

b = median of the first 3 measurements. Event = first measurement with median $D > 10b$ AND > 20. If it never fires → miss.

Our spec

A, B = 512 examples, with replacement, from the train split · K = 5 pairs → median + 25/75% · $\eta = 10^{-3}$ · plain gradient steps on train cross-entropy (no optimiser state) · all parameters · CPU float64 · every 100 epochs (401 checkpoints; 1 epoch = 1 step).

Your runs

$p = 113$ modular addition · 1-layer, 4-head transformer, 210,432 params · full-batch AdamW, 3,830 train examples · 5 seeds · grok epochs 12987, 6921, 9510, 11019, 23747 (test acc > 0.9).

PASS = all three

lead_frac ≥ 0.8 (event < grok − 100) · Spearman ρ (event, grok) ≥ 0.9 · 0.05 ≤ mean relative lead ≤ 0.9.

Plan

Freeze rules in context.md → write predictor → sanity checks (A = B → 0; halve η → same D; reproducible) → plug into run_nanda_benchmark.py & analyze_nanda_unified.py → run 5 seeds from checkpoints (minutes to tens of minutes) → look at raw curves → score → log weak points.

Watch out for

Short lead at lr 10^{-3} (Xu: 43%; 24% at 3×10^{-3}) · non-grokking runs also rise 30–50× · separation only in hindsight · no-weight-decay runs rise too · single-author preprints, no replication · floor 20 may not transfer · *(our reasoning)* $D \propto 1/\text{gradient size}$, so shrinking gradients alone can inflate D.

Sources

Xu (2026) arXiv:2602.16746 (main) · Xu (2026) arXiv:2602.16967 (percentiles only) · Smith, Dherin, Barrett & De (2021) ICLR, arXiv:2101.12176 (order matters at 2nd order).

References

[1] Y. Xu (2026). Low-Dimensional and Transversely Curved Optimization Dynamics in Grokking. arXiv:2602.16746. Code: github.com/skydancerosel/grokking-integrability.
[2] Y. Xu (2026). Early-Warning Signals of Grokking via Loss-Landscape Geometry. arXiv:2602.16967.
[3] S. L. Smith, B. Dherin, D. G. T. Barrett, S. De (2021). On the Origin of Implicit Regularization in Stochastic Gradient Descent. ICLR 2021. arXiv:2101.12176.